

D212: Data Mining II

Task 3: Association Rules & Lift Analysis

Hannah R. Bergman

WGU Email: hbergm3@wgu.edu

Part I: Research Question.....	2
A1. Research Question.....	2
A2. Goal of the Data Analysis.....	2
Part II: Market Basket Justification.....	2
B1. Market Basket Technique & Expected Outcomes.....	2
How Market Basket Analyzes Our Patient Dataset.....	2
Expected Outcomes of the Analysis.....	2
B2. Transactions Example.....	3
B3. Assumption of Market Basket Analysis.....	3
Part III: Data Preparation & Analysis.....	4
C1. Data Cleaning & Transformation.....	4
Data Set Transformation.....	4
Cleaned Data Set.....	7
C2. Code Demo Screenshots.....	8
Generating Association Rules with the Apriori Algorithm.....	8
C3. Association Rules Table: Support, Lift & Confidence Values.....	9
C4. Relevant Rules Generated by the Apriori Algorithm.....	10
Part IV: Data Summary & Implications.....	10
D1. Support, Lift & Confidence Significance Summary.....	10
D2. Practical Significance.....	11
D3. Recommended Course of Action.....	12
Part V: Attachments.....	12
E. Panopto Video Recording.....	12
F. Data & Code Sources.....	12
G. References.....	12

Part I: Research Question

A1. Research Question

What common combinations of prescriptions are frequently administered together among hospital patients, and what underlying patient characteristics or conditions might these prescription combinations suggest?

A2. Goal of the Data Analysis

The primary goal of this analysis is to identify the most frequently co-prescribed medications among patients using Market Basket Analysis, in order to uncover common treatment patterns and potential comorbidity groupings that can inform more targeted and cost-effective care strategies for our hospital.

Part II: Market Basket Justification

B1. Market Basket Technique & Expected Outcomes

How Market Basket Analyzes Our Patient Dataset

Market Basket Analysis (MBA), a data mining technique typically used in retail to understand purchasing patterns (TechTarget, 2023), will be applied to our patient prescription data, to help us uncover any significant associations between co-prescribed medications.

Each patient's medication list, coming from the columns 'Presc01' through 'Presc20', will be treated as a **"basket"** or transaction of **"items"** (prescription drug names). The Apriori algorithm will be used to identify frequent **"itemsets"**, or medication combinations, within these patient **"baskets."** By analyzing the **"itemsets"** we find, we can identify frequently co-prescribed medication combinations, revealing common treatment patterns and potential underlying health conditions relating to the use of the prescribed medications.

Association rules will then be generated to quantify the likelihood of medications being co-prescribed. MBA operates on an "if-then" (association rule) logic (GeeksforGeeks, 2024). For example, **if** Medication A is prescribed (antecedent), **then** Medication B is also likely to be prescribed (consequent). This approach shows us how the presence of one medication can predict the likelihood of others, providing valuable insights into prescribing practices. These rules will be evaluated using support (frequency of occurrence), confidence (probability of co-occurrence), and lift (strength of association beyond chance), metrics that allow us to assess the significance and practical clinical relevance of the patterns we find (Chaudhary, 2024).

Expected Outcomes of the Analysis

Our Market Basket Analysis should help us uncover key insights into patient prescription patterns, by identifying frequently co-prescribed medications to reveal standard treatment standards and potential comorbidity clusters.

We'll also generate association rules that quantify the likelihood of medication co-prescription. These rules can offer us a data-driven perspective on prescribing trends, identifying medication combinations indicative of specific treatment pathways.

These insights can then help support the development of bundled care plans, the customization of treatment protocols, and the implementation of more proactive and cost-effective patient care strategies for our hospital.

B2. Transactions Example

An example of a customer “basket” (transaction) is shown below:

```
# B2. Transaction Example
example_transaction = df.dropna(how='all').iloc[1]
example_transaction

Presc01          citalopram
Presc02          benicar
Presc03  amphetamine salt combo xr
Presc04          NaN
Presc05          NaN
Presc06          NaN
Presc07          NaN
Presc08          NaN
Presc09          NaN
Presc10          NaN
Presc11          NaN
Presc12          NaN
Presc13          NaN
Presc14          NaN
Presc15          NaN
Presc16          NaN
Presc17          NaN
Presc18          NaN
Presc19          NaN
Presc20          NaN
Name: 3, dtype: object
```

In the context of our analysis, the “basket” (transaction) refers to a single row (in this case, row 1 or `df.iloc[1]`) of our dataset representing all “items” (prescriptions) associated with a single patient. In this transaction example, the “items” are: **citalopram, benicar, and amphetamine salt combo xr**. Together, all three of these prescriptions (items) make up the “itemset”.

B3. Assumption of Market Basket Analysis

Market Basket Analysis operates on the assumption that the presence of certain items (or prescriptions, in our case) within a transaction is meaningfully associated with the presence of other items - meaning, items that frequently occur together are likely to be related in some way. However, MBA identifies correlations, not causations (Amruta, 2024). For example, if Medication A and Medication B are often prescribed together, MBA suggests a noteworthy pattern, but it doesn't imply that one medication causes the other to be prescribed.

This assumption allows MBA to uncover hidden patterns within data, but it also means our results need to be interpreted carefully, to avoid making unsupported conclusions about the relationships between items. This is particularly important in the context of my research question, as the second part seeks to explore what underlying patient characteristics or conditions might be suggested by the prescription combinations identified.

For instance, if Medication A (a heart medication used to prevent heart disease) and Medication B (an antihypertensive) frequently appear together, this pattern may suggest a shared underlying condition - such as cardiovascular disease requiring both treatments. However, MBA does not infer that Medication A leads to the prescription of Medication B. One patient may require both due to more advanced disease, while another may take only Medication A because they don't have high blood pressure.

Part III: Data Preparation & Analysis

C1. Data Cleaning & Transformation

Data Set Transformation

Step #1. Check dataset column data types (dtype), value counts (non-null count) and total number of rows (entries) and columns using the info() method.

```
# Review dataset column data types, value counts and total rows/columns
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15002 entries, 0 to 15001
Data columns (total 20 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Presc01     7501 non-null   object
1   Presc02     5747 non-null   object
2   Presc03     4389 non-null   object
3   Presc04     3345 non-null   object
4   Presc05     2529 non-null   object
5   Presc06     1864 non-null   object
6   Presc07     1369 non-null   object
7   Presc08     981 non-null    object
8   Presc09     654 non-null    object
9   Presc10     395 non-null    object
10  Presc11     256 non-null    object
11  Presc12     154 non-null    object
12  Presc13     87 non-null     object
13  Presc14     47 non-null     object
14  Presc15     25 non-null     object
15  Presc16     8 non-null      object
16  Presc17     4 non-null      object
17  Presc18     4 non-null      object
18  Presc19     3 non-null      object
19  Presc20     1 non-null      object
dtypes: object(20)
memory usage: 2.3+ MB
```

Step #2. Inspect and examine the newly created DataFrame (df) to identify any apparent high-level issues.

```
# Preview the dataframe (displaying all columns)
pd.set_option("display.max_columns", None)
df.head()
```

	Presc01	Presc02	Presc03	Presc04	Presc05	Presc06	Presc07	Presc08	Presc09	Presc10	Presc11	Presc12	Presc13
0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
1	amlodipine	albuterol aerosol	allopurinol	pantoprazole	lorazepam	omeprazole	mometasone	fluconazole	gabapentin	pravastatin	cialis	losartan	metoprolol succinate su XL
2	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
3	citalopram	benicar	amphetamine salt combo xr	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

It appears that every other row in our DataFrame is filled with NaN values, likely due to irregular formatting during data import (likely blank lines in the CSV). We'll drop these rows where all prescription columns are NaN, using the dropna() method, saving this change to a new DataFrame (df_clean).

```
# Remove empty rows
df_clean = df.dropna(how='all')

# Preview the clean dataframe
df_clean.head()
```

	Presc01	Presc02	Presc03	Presc04	Presc05	Presc06	Presc07	Presc08	Presc09	Presc10	Presc11	Presc12	Presc13
1	amlodipine	albuterol aerosol	allopurinol	pantoprazole	lorazepam	omeprazole	mometasone	fluconazole	gabapentin	pravastatin	cialis	losartan	metoprolol succinate XL
3	citalopram	benicar	amphetamine salt combo xr	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
5	enalapril	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
7	paroxetine	allopurinol	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
9	abilify	atorvastatin	folic acid	naproxen	losartan	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

Step #3. Create a list-of-lists (temp_list), where each inner list contains the items (prescriptions) associated with a single transaction (patient), showing a preview of the list by displaying the first

3 entries and last 3 entries.

```
# Convert the DataFrame to a list of lists, skipping NaN values
temp_list = [
    [str(item) for item in row if pd.notnull(item)]
    for row in df_clean.values
]

# Show first 3 list entries
for i in range(3):
    print(f"Index {i}: {temp_list[i]}")

print("...")

# Show last 3 list entries
for i in range(-3, 0):
    print(f"Index {len(temp_list) + i}: {temp_list[i]}")

Index 0: ['amlodipine', 'albuterol aerosol', 'allopurinol', 'pantoprazole', 'lorazepam', 'omeprazole', 'mometason
e', 'fluconazole', 'gabapentin', 'pravastatin', 'cialis', 'losartan', 'metoprolol succinate XL', 'sulfamethoxazol
e', 'abilify', 'spironolactone', 'albuterol HFA', 'levofloxacin', 'promethazine', 'glipizide']
Index 1: ['citalopram', 'benicar', 'amphetamine salt combo xr']
Index 2: ['enalapril']
...
Index 7498: ['clopidoagrel']
Index 7499: ['alprazolam', 'losartan']
Index 7500: ['amphetamine salt combo xr', 'levofloxacin', 'diclofenac sodium', 'cialis']
```

Market Basket Analysis requires our data to be structured in a way that reflects individual transactions. In our case, each row (patient) represents one “basket” or transaction, and the “items” in that basket are their prescriptions. Creating a list-of-lists, as we did above, provides the structure our Apriori algorithm specifically needs to easily process and identify frequent “itemsets” (prescription combinations) and generate association rules.

Step #4. Encode the list of prescriptions (transaction) into one-hot encoded format.

```
# Encode the list of prescriptions (transactions) into one-hot encoded format
encoder = TransactionEncoder()
encoded_array = encoder.fit_transform(temp_list)

# Create a DataFrame from the encoded array
df_cleaned = pd.DataFrame(encoded_array, columns=encoder.columns_)

# Preview the one-hot encoded dataframe
df_cleaned.head()
```

	Duloxetine	Premarin	Yaz	abilify	acetaminophen	actonel	albuterol HFA	albuterol aerosol	alendronate	allopurinol	alprazolam	amitriptyline	amlodipine	amoxicil
0	False	False	False	True	False	False	True	True	False	True	False	False	True	Fal
1	False	False	False	False	False	False	False	False	False	False	False	False	False	Fal
2	False	False	False	False	False	False	False	False	False	False	False	False	False	Fal
3	False	False	False	False	False	False	False	False	False	True	False	False	False	Fal
4	False	False	False	True	False	False	False	False	False	False	False	False	False	Fal

Taking our list-of-lists, we transform (fit) it into a binary matrix (one-hot encoded) format using TransactionEncoder. This creates a new encoded array, which we’ll create a new DataFrame from (df_cleaned). Within this DataFrame, our columns now represent the prescriptions and each row shows whether a patient has received them or not (True or False).

Step #5. Double-check the transformed dataset column data types, value counts and total number of rows (entries) and columns using the info() method, again, to ensure it has been

encoded properly (all of the dtypes, at this point, should be bool - True/False values).

```
# Review transformed dataset column data types, value counts and total rows/columns
df_cleaned.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7501 entries, 0 to 7500
Columns: 119 entries, Duloxetine to zolpidem
dtypes: bool(119)
memory usage: 871.8 KB
```

Cleaned Data Set

The cleaned dataset, '**d212_task3_transformed.csv**', can be found within the main project directory and is also included as an attachment to this report.

```
# Save the transformed dataset as a CSV file
df_cleaned.to_csv('d212_task3_transformed.csv', index=False)

print(f"The final cleaned and scaled dataset has been saved as:\n 'd212_task3_transformed.csv'")
```

```
The final cleaned and scaled dataset has been saved as:
'd212_task3_transformed.csv'
```

C2. Code Demo Screenshots

Generating Association Rules with the Apriori Algorithm

Step #1. Generate frequent itemsets using the Apriori algorithm.

```
# Generate frequent itemsets using the Apriori algorithm
frequent_itemsets = apriori(df_cleaned, min_support=0.05, use_colnames=True)
frequent_itemsets
```

	support	itemsets
0	0.238368	(abilify)
1	0.079323	(alprazolam)
2	0.071457	(amlodipine)
3	0.068391	(amphetamine salt combo)
4	0.179709	(amphetamine salt combo xr)
5	0.129583	(atorvastatin)
6	0.174110	(carvedilol)
7	0.076523	(cialis)
8	0.087188	(citalopram)
9	0.059992	(clopidogrel)
10	0.081056	(dextroamphetamine XR)
11	0.163845	(diazepam)
12	0.095054	(doxycycline hyclate)
13	0.080389	(ezetimibe)
14	0.051060	(fenofibrate)
15	0.052393	(furosemide)
16	0.065858	(glipizide)
17	0.170911	(glyburide)
18	0.063325	(levofloxacin)
19	0.098254	(lisinopril)
20	0.132116	(losartan)
21	0.050527	(metformin)
22	0.095321	(metoprolol)
23	0.058526	(naproxen)
24	0.062525	(paroxetine)
25	0.050927	(abilify, amphetamine salt combo xr)
26	0.059725	(carvedilol, abilify)
27	0.052660	(diazepam, abilify)

I used the apriori algorithm from the mlxtend library to find itemsets with a minimum support threshold of 0.05. This means that the itemset must be present in at least 5% of all transactions. Research suggests that this threshold range (0.05 to 0.2) is ideal for small-to-medium datasets like ours (GeeksforGeeks, 2025). Through experimentation, I found that thresholds between 0.06 and 0.2 produced frequent itemsets. However, none of these itemsets met the requirements for generating association rules, resulting in empty association rule tables. Even with adjustments as low as 0.01, while more itemsets were generated (including rarer co-prescriptions), the computation was slower and included more statistically insignificant noise. Ultimately, I found a minimum support threshold of 0.05 was optimal. Although only 27 itemsets were generated, they represent the most frequent and likely meaningful prescription combinations.

Step #2. Generate association rules from the frequent itemsets.

```
# Generate association rules from the frequent itemsets
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=1.1)
rules
```

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	representativity	leverage	conviction	zhangs_metric	jaccard	ce
0	(amphetamine salt combo xr)	(abilify)	0.179709	0.238368	0.050927	0.283383	1.188845	1.0	0.008090	1.062815	0.193648	0.138707	0.0
1	(abilify)	(amphetamine salt combo xr)	0.238368	0.179709	0.050927	0.213647	1.188845	1.0	0.008090	1.043158	0.208562	0.138707	0.0
2	(carvedilol)	(abilify)	0.174110	0.238368	0.059725	0.343032	1.439085	1.0	0.018223	1.159314	0.369437	0.169312	0.0
3	(abilify)	(carvedilol)	0.238368	0.174110	0.059725	0.250559	1.439085	1.0	0.018223	1.102008	0.400606	0.169312	0.0
4	(diazepam)	(abilify)	0.163845	0.238368	0.052660	0.321400	1.348332	1.0	0.013604	1.122357	0.308965	0.150648	0.0
5	(abilify)	(diazepam)	0.238368	0.163845	0.052660	0.220917	1.348332	1.0	0.013604	1.073256	0.339197	0.150648	0.0

The association rules were generated using the `association_rules()` function from the `mlxtend` library. These rules are derived from the frequent itemsets discovered by the apriori algorithm and are based on lift, which is a metric that compares the likelihood of prescriptions appearing together with the probability of them being independent (Ali, 2023). Because our research question's main objective is to find common prescription combinations, we're particularly interested in frequent, significant co-occurrences. Choosing lift helps to ensure that our frequent itemsets are not merely coincidental pairings of common medications, but rather that their co-occurrence is greater than would be expected by chance. As a result, basing our association rules on lift makes sure that any discovered associations are genuine and not the result of random chance. Since a value of 1.0 denotes no association, I established a minimum lift value threshold of 1.1. A lift value greater than 1.0 signifies a positive association between co-prescriptions, with higher values indicating a stronger association.

C3. Association Rules Table: Support, Lift & Confidence Values

```
# Generate association rules from the frequent itemsets
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=1.1)
rules
```

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	representativity	leverage	conviction	zhangs_metric	jaccard	ce
0	(amphetamine salt combo xr)	(abilify)	0.179709	0.238368	0.050927	0.283383	1.188845	1.0	0.008090	1.062815	0.193648	0.138707	0.0
1	(abilify)	(amphetamine salt combo xr)	0.238368	0.179709	0.050927	0.213647	1.188845	1.0	0.008090	1.043158	0.208562	0.138707	0.0
2	(carvedilol)	(abilify)	0.174110	0.238368	0.059725	0.343032	1.439085	1.0	0.018223	1.159314	0.369437	0.169312	0.0
3	(abilify)	(carvedilol)	0.238368	0.174110	0.059725	0.250559	1.439085	1.0	0.018223	1.102008	0.400606	0.169312	0.0
4	(diazepam)	(abilify)	0.163845	0.238368	0.052660	0.321400	1.348332	1.0	0.013604	1.122357	0.308965	0.150648	0.0
5	(abilify)	(diazepam)	0.238368	0.163845	0.052660	0.220917	1.348332	1.0	0.013604	1.073256	0.339197	0.150648	0.0

This association rules table analyzes medication pairings among hospital patients and identifies positively correlated co-prescriptions using support, confidence, and lift metrics. For example, the rule *(carvedilol) ⇒ (abilify)* suggests that patients prescribed Carvedilol are ~1.44 (1.4390) times more likely to also be prescribed Abilify than would be expected by chance; this association has a support of ~5.97% (0.0597) and a confidence of ~34.3% (0.3430). While all

identified medication pairs demonstrate a positive association (lift values exceeding 1.0, as we established by setting our minimum lift value threshold to 1.1 earlier), the relatively low support values we see indicate that these co-prescriptions, although statistically significant, occur infrequently within the dataset.

C4. Relevant Rules Generated by the Apriori Algorithm

```
# Display the top 3 rules (based on lift)
top_rules = rules.sort_values(by='lift', ascending=False).head(3)
top_rules
```

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	representativity	leverage	conviction	zhangs_metric	jaccard	cert
2	(carvedilol)	(abilify)	0.174110	0.238368	0.059725	0.343032	1.439085	1.0	0.018223	1.159314	0.369437	0.169312	0.13
3	(abilify)	(carvedilol)	0.238368	0.174110	0.059725	0.250559	1.439085	1.0	0.018223	1.102008	0.400606	0.169312	0.05
4	(diazepam)	(abilify)	0.163845	0.238368	0.052660	0.321400	1.348332	1.0	0.013604	1.122357	0.308965	0.150648	0.10

Rule #1 (Index 2): *(carvedilol) ⇒ (abilify)*
If a patient is prescribed *carvedilol*, then they are also likely to be prescribed *abilify*.

2	(carvedilol)	(abilify)	0.174110	0.238368	0.059725	0.343032	1.439085	1.0	0.018223	1.159314	0.369437	0.169312	0.13
---	--------------	-----------	----------	----------	----------	----------	----------	-----	----------	----------	----------	----------	------

Rule #2 (Index 3): *(abilify) ⇒ (carvedilol)*
If a patient is prescribed *abilify*, then they are also likely to be prescribed *carvedilol*.

3	(abilify)	(carvedilol)	0.238368	0.174110	0.059725	0.250559	1.439085	1.0	0.018223	1.102008	0.400606	0.169312	0.05
---	-----------	--------------	----------	----------	----------	----------	----------	-----	----------	----------	----------	----------	------

Rule #3 (Index 4): *(diazepam) ⇒ (abilify)*
If a patient is prescribed *diazepam*, then they are also likely to be prescribed *abilify*.

4	(diazepam)	(abilify)	0.163845	0.238368	0.052660	0.321400	1.348332	1.0	0.013604	1.122357	0.308965	0.150648	0.10
---	------------	-----------	----------	----------	----------	----------	----------	-----	----------	----------	----------	----------	------

The top three association rules based on lift reveal strong positive relationships between certain co-prescribed medications. The rule *(carvedilol) ⇒ (abilify)* shows a confidence of ~34.3% and a lift of ~1.44, indicating that Carvedilol users are significantly more likely to also be prescribed Abilify than expected by chance. The inverse rule *(abilify) ⇒ (carvedilol)* similarly reflects a notable relationship, though with slightly lower confidence (~25%). Additionally, *(diazepam) ⇒ (abilify)* demonstrates a ~32.1% confidence and a lift of ~1.35, suggesting a meaningful co-prescription pattern. These patterns may reflect underlying comorbidities or shared treatment pathways worth exploring further in the context of patient conditions.

Part IV: Data Summary & Implications

D1. Support, Lift & Confidence Significance Summary

From the analysis, the metrics support, confidence, and lift provide meaningful insight into the strength and relevance of medication co-prescription patterns in the dataset.

- Support** quantifies how frequently a specific combination of medications appears across all patient records. The top rules had support values between ~5-6%, indicating these prescription combinations appear in a noteworthy portion of patient records. For example, *(abilify, carvedilol)* appears in ~5.97% of cases - frequent enough to suggest a

common treatment pattern, suggesting they serve a routine or clinically significant role in treatment plans.

- **Lift** is one of the most powerful indicators in this analysis. It reflects how much more likely two medications are to be prescribed together compared to if their co-occurrence were completely random. All top rules had lift values greater than 1.3 to 1.4, indicating strong positive associations. (*carvedilol*) $\Rightarrow$ (*abilify*) and (*abilify*) $\Rightarrow$ (*carvedilol*) both had a lift of ~ 1.44 , while (*diazepam*) $\Rightarrow$ (*abilify*) had a lift of ~ 1.35 . These values confirm that the relationships are not just coincidental. These medications are prescribed together more frequently than random chance would suggest, reflecting a clinical or treatment-related connection.
- **Confidence** measures how reliably one medication is prescribed given that another is already prescribed. The rule (*carvedilol*) $\Rightarrow$ (*abilify*) has confidence of $\sim 34.3\%$, meaning about over one-third of patients on carvedilol are also prescribed Abilify. Similarly, (*diazepam*) $\Rightarrow$ (*abilify*) has a confidence of $\sim 32.1\%$. This suggests potential shared patient profiles or comorbidities. Patients on Carvedilol, which is a beta blocker for heart conditions (NHS, 2024), may frequently be treated with Abilify, an antipsychotic (Cleveland Clinic, 2024) - possibly due to co-occurring cardiovascular and psychiatric conditions. Confidence helps us understand directionality - which drug is likely to be added to a regimen if another is already prescribed.

Circling back to my initial research question, our analysis reveals that certain prescription combinations (like Abilify with Carvedilol or Diazepam) are not only frequently co-prescribed, but also strongly associated, occurring together significantly more often than expected.

These findings also suggest the presence of shared underlying patient characteristics or conditions. For instance, co-prescription of Carvedilol (cardiovascular) and Abilify (psychiatric) may indicate a subset of patients dealing with both heart-related conditions and mental health disorders. Similarly, the frequent pairing of Diazepam, which is typically used to treat anxiety disorders (NHS, 2022), and Abilify might reflect treatment plans involving anxiety, mood stabilization, or comorbid psychiatric diagnoses.

By using support to confirm relevance, confidence to assess predictability, and lift to validate non-random associations, our analysis successfully answers our initial research question and identifies prescription patterns that warrant further clinical interpretation, potentially guiding insights into multimorbidity profiles and treatment strategies among our patients.

D2. Practical Significance

The analysis revealed that prescription pairs like Abilify + Carvedilol and Abilify + Diazepam were frequently prescribed together and strongly associated. They occurred approximately 30–34% of the time when either medication was present and appeared together more than 40% more often than expected by chance.

These findings can help identify multimorbidity patterns and common comorbidities in our hospital population, such as the suggested co-occurrence of psychiatric disorders and cardiovascular conditions. Understanding these patterns can allow our patient care teams to proactively monitor and manage these types of overlapping conditions, supporting more holistic patient management.

Additionally, identifying strong medication associations can help refine clinical decision support

systems by flagging patients for potential review when certain prescription combinations are considered. For example, if a patient is prescribed Abilify and has cardiovascular risk factors, their care team could be prompted to consider cardiac medications like Carvedilol for potential inclusion (or safety concerns).

Lastly, patterns in co-prescribing can inform pharmacy stocking, interdisciplinary care planning, and resource prioritization across departments. Our hospital administrators and clinical teams can also use this information to design bundled care protocols or patient pathways for common medication combinations, which can aid resource allocation and care coordination efforts.

D3. Recommended Course of Action

To extend the value of our findings, we can join prescription data with patient characteristic data. Merging our rule results with patient demographic or diagnostic data (e.g., comorbidities, age groups) can help us determine who is most likely to receive each prescription combination and what conditions these combinations suggest to enhance the clinical interpretability of the rules we generated.

Based on previous research we've performed, we may also want to consider clustering or segmenting patients based on prescriptions, using unsupervised learning (k-means or hierarchical clustering) on prescription patterns to identify distinct patient subgroups with similar medication profiles to help explore treatment archetypes or more complex care cases to further improve clinical insights.

Part V: Attachments

E. Panopto Video Recording

Panopto video recording:

<https://wgu.hosted.panopto.com/Panopto/Pages/Viewer.aspx?id=0e09c0e0-e8a3-4a45-b389-b2b201227721>

F. Data & Code Sources

Western Governors University. (2023). *Medical Data and Dictionary Files* [Dataset]. Data Files and Associated Dictionary Files.

Retrieved March 21, 2025, from

<https://access.wgu.edu/ASP3/aap/content/g9rke9s0rlc9ejd92md0.html>

G. References

Ali, M. (2023, January 23). *Association Rule Mining in Python Tutorial*. Datacamp.

Retrieved March 26, 2025 from

datacamp.com/tutorial/association-rule-mining-python

Amruta. (2024, December 1). *Market Basket Analysis: A Comprehensive Guide for Business*. Analytics Vidhya.

Retrieved March 24, 2025 from

- analyticsvidhya.com/blog/2021/10/a-comprehensive-guide-on-market-basket-analysis
Chaudhary, S. (2024, January 29). *Market Basket Analysis: Anticipating Customer Behavior*. Turing.
Retrieved March 24, 2025, from
turing.com/kb/market-basket-analysis
- Cleveland Clinic. (2024, December 11). *Aripiprazole Tablets*. Cleveland Clinic.
Retrieved March 26, 2025 from
clevelandclinic.org/health/drugs/19695-aripiprazole-tablets
- GeeksforGeeks. (2025, January 15). *Apriori Algorithm*. GeeksforGeeks.
Retrieved March 25, 2025 from
geeksforgeeks.org/apriori-algorithm
- GeeksforGeeks. (2024, August 16). *Market Basket Analysis in Data Mining*. GeeksforGeeks.
Retrieved March 24, 2025 from
geeksforgeeks.org/market-basket-analysis-in-data-mining
- NHS. (2024, July 5). *About Carvedilol*. Medicines A to Z.
Retrieved March 24, 2025 from
nhs.uk/medicines/carvedilol/about-carvedilol
- NHS. (2022, February 3). *About Diazepam*. Medicines A to Z.
Retrieved March 26, 2025 from
nhs.uk/medicines/diazepam/about-diazepam
- TechTarget. (2023, January 31). *Market Basket Analysis*. TechTarget.
Retrieved March 24, 2025 from
techtarget.com/searchcustomerexperience/definition/market-basket-analysis